

An error propagation operator

Document #: P2561R1
Date: 2022-10-11
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Preface](#)
- [3 Introduction](#)
- [4 An automatic propagation operator](#)
 - [4.1 Syntax for C++](#)
 - [4.1.1 Why not try?](#)
 - [4.1.2 Why not prefix?](#)
 - [4.1.3 ?? in other languages](#)
 - [4.2 Semantics](#)
 - [4.2.1 Lifetime](#)
 - [4.2.2 decltype](#)
 - [4.2.3 requires](#)
 - [4.2.4 Short-circuiting fold](#)
 - [4.2.5 Range of expected to expected of Range](#)
 - [4.2.6 Naming](#)
 - [4.3 Potential directions to go from here](#)
 - [4.3.1 Error continuations](#)
 - [4.3.2 Not propagating errors](#)
- [5 References](#)

§ 1 Revision History

The title of [P2561R0] was `operator??`, but isn't actually proposing that token, so it's not the best title. Likewise, `try_traits` is a bad name for the collection of functionality for the same reason that the paper described `try` as being a bad spelling for the operator. `is_ok` has been renamed to `has_value`, since that's actually what we name that facility everywhere. A few other details added in addition to the two renames.

§ 2 Preface

It is important to clarify a few things up front. It is not the position of this paper that exceptions are bad. Or that exceptions are good. It is not the goal of this paper to convince you to start using exceptions, nor is it to convince you to stop using exceptions.

This paper simply recognizes that there are many code bases (or parts thereof) that do not use exceptions and probably will not in the future. That could be for performance or space reasons. It could be because exceptions are unsupported on a particular platform. It could be for code understandability reasons. Regardless, some code bases do not use exceptions. Moreover, some problems are not solved well by exceptions – even in code bases that otherwise use them to solve problems that they are more tailored to solve.

The problem is, C++ does not currently have a good story for error handling without exceptions. We're moving away from returning `bool` or error codes in favor of solutions like `std::expected` ([P0323R12]), but the ergonomics of such types are not there yet. Bad ergonomics leads to code that is clunkier than it needs to be, harder to follow, and, significantly and ironically, error-prone.

We should try to improve such uses too.

§ 3 Introduction

Let's start with a fairly small example of a series of functions that can generate errors, but don't themselves handle them – they just need to propagate them up. With exceptions, this might look like:

```
auto foo(int i) -> int; // might throw an E
auto bar(int i) -> int; // might throw an E
```

```

auto strcat(int i) -> std::string {
    int f = foo(i);
    int b = bar(i);
    return std::format("{}{}", f, b);
}

```

There's a lot to like about exceptions. One nice advantage is the zero syntactic overhead necessary for propagating errors. Errors just propagate. You don't even have to know which functions can fail.

We don't even need to declare variables to hold the results of `foo` and `bar`, we can even use those expressions inline, knowing that we'll only call `format` if neither function throws an exception:

```

auto foo(int i) -> int; // might throw an E
auto bar(int i) -> int; // might throw an E

auto strcat(int i) -> std::string {
    return std::format("{}{}", foo(i), bar(i));
}

```

But with the newly adopted `std::expected<T, E>`, it's not quite so nice:

```

auto foo(int i) -> std::expected<int, E>;
auto bar(int i) -> std::expected<int, E>;

auto strcat(int i) -> std::expected<std::string, E>
{
    auto f = foo(i);
    if (not f) {
        return std::unexpected(f.error());
    }

    auto b = bar(i);
    if (not b) {
        return std::unexpected(b.error());
    }

    return std::format("{}{}", *f, *b);
}

```

This is significantly longer and more tedious because we have to do manual error propagation. This manual error propagation is most of the code in this short example, and is bad not just because of the lengthy boilerplate, but also because:

- we're giving a name, `f`, to the `expected` object, not the success value. The error case is typically immediately handled, but the value case could be used multiple times and now has to be used as `*f` (which is pretty weird for something that is decidedly not a pointer or even, unlike iterators, a generalization of pointer) or `f.value()`
- the “nice” syntax for propagation - `return std::unexpected(e)` - is inefficient - if `E` is something more involved than `std::error_code`, we really should `std::move(f).error()` into that. And even then, we're moving the error twice when we optimally could move it just once. The ideal would be: `return {std::unexpected, std::move(f).error()};`, which is something I don't expect a lot of people to actually write.

In an effort to avoid... that... many libraries or code bases that use this sort approach to error handling provide a macro, which usually looks like this ([Boost.LEAF](#), [Boost.Outcome](#), [mediapipe](#), [SerenityOS](#), etc. Although not all do, neither `folly's fb::Expected` nor `tl::expected` nor `llvm::Expected` provide such):

```

auto strcat(int i) -> std::expected<std::string, E>
{
    SOMETHING_TRY(int f, foo(i));
    SOMETHING_TRY(int b, bar(i));
    return std::format("{}{}", f, b);
}

```

Which avoids all those problems, though each such library type will have its own corresponding macro. Also these `TRY` macros (not all of them have `TRY` in the name) need to be written on their own line, since they are declarations - thus the one-line version of `strcat` in the exception version isn't possible.

Some more adventurous macros take advantage of the statement-expression extension, which would allow you to do this:

```

auto strcat(int i) -> std::expected<std::string, E>
{
    int f = SOMETHING_TRY_EXPR(foo(i));
    int b = SOMETHING_TRY_EXPR(bar(i));
    return std::format("{}{}", f, b);
}

```

And thus also write both macros inline. But this relies on compiler extensions, and this particular extension isn't quite as efficient as it could be - and in particular it doesn't move when it should.

Both macros also suffer when the function in question returns `expected<void, E>`, since you cannot declare (or assign to) a variable to hold that value, so the macro needs to emit different code to handle this case ([Boost.LEAF](#), [Boost.Outcome](#),¹ etc.)

To that end, in search for nice syntax, some people turn to coroutines:

```
auto strcat(int i) -> std::expected<std::string, E>
{
    int f = co_await foo(i);
    int b = co_await bar(i);
    co_return std::format("{}{}", f, b);

    // ... or
    co_return std::format("{}{}", co_await foo(i), co_await bar(i));
}
```

This can be made to work in a fully-conformant way (at the syntactic cost of having to now write `co_return`), and we can use the same syntax for both the `void` and non-`void` cases.

However, currently even the simple cases allocate which make this approach unusable in many production contexts. The coroutine machinery also isn't fully composable and runs into problems once you start doing something like `optional<expected<T, E>>` (or vice versa) or `task<optional<T>>`.

Which means the best-case today still involves being jealous of exceptions macros.

§ 4 An automatic propagation operator

Let's talk about Rust.

Rust's primary form of error handling is a sum type named `Result<T, E>`. Taking our original example here and rewriting it in Rust (as one does) would look like this:

Rust	C++
<pre>fn strcat(i: i32) -> Result<String, E> { let f = match foo(i) { Ok(i) => i, Err(e) => return Err(e), }; let b = match bar(i) { Ok(i) => i, Err(e) => return Err(e), } Ok(format!("{}", f, b)) }</pre>	<pre>auto strcat(int i) -> std::expected<std::string, E> { auto f = foo(i); if (not f) { return std::unexpected(f.error()); } auto b = bar(i); if (not b) { return std::unexpected(b.error()); } return std::format("{}{}", *f, *b); }</pre>

This fully manual version is already better than the C++ version due to pattern matching's ability to just give a name to the thing we care about (the value) and avoid giving a name to the thing we don't care about (the `Result` object).

But this isn't the way you do things in Rust.

Originally, there was a [try! macro](#) which was defined mostly as that match expression I have above. But then this got generalized into `operator?`, whose behavior is driven by the [Try trait](#) (originally there was `try-v1`, now this is `try-v2`). That allows simply writing this:

Rust	C++ with exceptions
<pre>fn strcat(i: i32) -> Result<String, E> { let f = foo(i)?; let b = bar(i)?; Ok(format!("{}", f, b)) // ... or simply ... Ok(format!("{}", foo(i)?, bar(i)?)) }</pre>	<pre>auto strcat(int i) -> std::string { int f = foo(i); int b = bar(i); return std::format("{}{}", f, b); // ... or simply ... return std::format("{}{}", foo(i), bar(i)); }</pre>

Now, Rust still has manual error propagation, but it's the minimal possible syntactic overhead: one character per expression.

Importantly, one character per expression is still actually an enormous amount more overhead than zero characters per expression, since that implies that you cannot have error-neutral functions - they have to manually propagate errors too.

But to those people who write code using types like `std::expected` today, who may use the kinds of macros I showed earlier or foray into coroutines, this is kind of a dream?

§ 4.1 Syntax for C++

Before diving too much into semantics, let's just start by syntax. Unfortunately, C++ cannot simply grab the Rust syntax of a postfix `?` here, because we also have the conditional operator `?:`, with which it can be ambiguous:

```
auto res = a ? * b ? * c : d;
```

That could be parsed two ways:

```
auto res1 = a ? (*(b?) * c) : d;
auto res2 = ((a?) * b) ? (*c) : d;
```

What if you assume that a `?` is a conditional operator and try to parse that until it fails, then back up and try again to parse a postfix `?` operator? Is that really a viable strategy? If we assume both `?`s are the beginning of a conditional, then that will eventually fail since we hit a `;` before a second `:` - but it's the outer `?` that failed, not the inner - do we retry the inner first (which would lead to the `res1` parse eventually) or the outer first (which would lead to the `res2` one)?

Maybe this is doable with parsing heroics, but at some point I have to ask if it's worth it.

Another reason that a single `?` might not be a good idea, even if it were possible to parse, would be [optional chaining](#). With that facility, if `o` were an `optional<string>`, `o?.size()` would be an `optional<size_t>` (that is either engaged with the original string's size, or empty). But if `o?` propagated the error, then `o?.size()` would itself be a valid expression that is a `size_t` (the string's size, and if we didn't have a string we would have returned). So if we want to support error continuations, we'd need distinct syntax for these cases.

This paper proposes an alternative token that isn't valid in C++ today, requires no parsing heroics, and doesn't conflict with a potential optional chaining operator: `??`

This is only one character longer, and just as questioning. It's easily unambiguous by virtue of not even being a valid token sequence today. But it's worth commenting further on this choice of syntax.

§ 4.1.1 Why not `try`?

For those libraries that provide this operation as a macro, the name is usually `TRY` and [\[P0779R0\]](#) previously suggested this sort of facility under the name `operator try`. As mentioned, Rust previously had an error propagation macro named `try!` and multiple other languages have such an error propagation operator ([Zig](#), [Swift](#), [Midori](#), etc.).

The problem is, in C++, `try` is strongly associated with *exceptions*. That's what a `try` block is for: to catch exceptions. In [\[P0709R4\]](#), there was a proposal for a `try` expression (in §4.5.1). That, too, was tied in with exceptions. Not only for us is it tied into exceptions, but it's used to *not* propagate the exception - `try` blocks are for handling errors.

Having a facility for error propagation in C++ which has nothing to do with exceptions still use the keyword `try` and do the opposite of a what a `try` block does today (i.e. propagate the error, instead of handling it) would be, I think, quite misleading. And the goal here isn't to interact with exceptions at all - it's simply to provide automated error propagation for those error handling cases that *don't* use exceptions.

§ 4.1.2 Why not prefix?

Once we settle on some punctuator, there's the question of whether this punctuator should be used as a prefix operator or a postfix operator. As prefix, there is no ambiguity with `?` at least, so we could use a more straightforward token. But I think postfix is quite a bit better. Consider the following example:

```
struct U { ... };

struct T {
    auto next() -> std::expected<U, E>;
};

auto lookup() -> std::expected<T, E>;

auto func() -> std::expected<U, E> {
    // as postfix
    U u = lookup()???.next()??;
```

```

// using the monadic operations
U u = lookup().and_then(&T::next);

// as prefix
U u = ?(?lookup()).next();

do_something_with(u);

return u;
}

```

The postfix version chains in a way that is quite easy to read.

Using the monadic operations ([P2505R4]) is fine, they're nice in this case (which is basically optimal for them) but they tend to be quite tedious once you stray from this exact formulation (e.g. if `T::next()` took another argument).

The prefix version is borderline illegible.

Even if we consider only one or the other side of the member access as needing propagation:

- Accessing a member after propagating: `x?? .y` vs `(?x) .y`
- Propagating after accessing a member: `x .y??` vs `?x .y` or `?(x .y)`

The postfix operator is quite a bit easier to understand, since it's always right next to the expression that is potentially the error.

§ 4.1.3 ?? in other languages

`??` is called a “null (or nil) coalescing operator” in some languages (like C# or JavaScript or Swift) where `x ?? y` is roughly equivalent to what C++ would spell as `x ? *x : y` except that `x` is only evaluated once. Kotlin spells this operator `?:`, but it behaves differently from the gcc extension since `x ?: y` in gcc evaluates as `x ? x : y` rather than `x ? *x : y`.

For `x` being some kind of `std::optional<T>` or `std::expected<T, E>`, this can *mostly* already be spelled `x.value_or(y)`. The difference is that here `y` is unconditionally evaluated, which is why [P2218R0] proposes a separate `opt.value_or_else(f)` which invokes `f`. Which would make a proper equivalence be spelled `x.value_or_else([&]{ return y; })`.

I'm not aware of any proposals to add this particular operator in C++, but because we already have two types that directly provide that functionality (as would many other non-std flavors thereof), and because it's fairly straightforward to write such an algorithm generically, it wouldn't seem especially valuable to have a dedicated operator for this functionality – so it's probably safe to take for this use-case.

It certainly would be nice to have both, but given a choice between a null coalescing operator and an error propagation one, I'd choose the latter.`

Of course, now we have to talk about semantics.

§ 4.2 Semantics

This paper suggests that `??` evaluate roughly as follows:

```

auto strcat(int i) -> std::expected<std::string, E>
{

    int f = foo(i)??;

    int b = bar(i)??;
}

```

```

auto strcat(int i) -> std::expected<std::string, E>
{
    using _Return = std::error_propagation_traits<
        std::expected<std::string, E>>;

    auto&& __f = foo(i);
    using _TraitsF = std::error_propagation_traits<
        std::remove_cvref_t<decltype(__f)>>;
    if (not _TraitsF::has_value(__f)) {
        return _Return::from_error(
            _TraitsF::extract_error(FWD(__f)));
    }
    int f = _TraitsF::extract_value(FWD(__f));

    auto&& __b = bar(i);
    using _TraitsB = std::error_propagation_traits<
        std::remove_cvref_t<decltype(__b)>>;
    if (not _TraitsB::has_value(__b)) {
        return _Return::from_error(
            _TraitsB::extract_error(FWD(__b)));
    }
    int b = _TraitsB::extract_value(FWD(__b));
}

```

```

    return std::format("{}{}", f, b);
}

```

```

    return std::format("{}{}", f, b);
}

```

The functionality here is driven by a new traits type called `std::error_propagation_traits`, such that a given specialization supports:

- telling us when the object is truthy: `has_value`
- extracting a value (`extract_value`) or error (`extract_error`) from it
- constructing a new object from either a value (`from_value`, not necessary in the above example, but will demonstrate a use later) or an error (`from_error`)

Note that this does not support deducing return type, since we need the return type in order to know how construct it - the above desugaring uses the return type of `std::expected<std::string, E>` to know how to re-wrap the potential error that `foo(i)` or `bar(i)` could return. This is important because it avoids the overhead that nicer syntax like `std::unexpected` or `outcome::failure` introduces (neither of which allow for deducing return type anyway, at least unless the function unconditionally fails), while still allowing nicer syntax.

This isn't really a huge loss, since in these contexts, you can't really deduce the return type anyway - since you'll have some error type and some value type. So this restriction isn't actually restrictive in practice.

These functions are all very easy to implement for the kinds of types that would want to support a facility like `??`. Here are examples for `optional` and `expected` (with `constexpr` omitted to fit):

```

template <class T>
struct error_propagation_traits<optional<T>> {
    using value_type = T;
    using error_type = nullopt_t;

    auto has_value(optional<T> const& o) -> bool {
        return o.has_value();
    }

    // extractors
    auto extract_value(auto&& o) -> auto&& {
        return *FWD(o);
    }
    auto extract_error(auto&&) -> error_type {
        return nullopt;
    }

    // factories
    auto from_value(auto&& v) -> optional<T> {
        return optional<T>(in_place, FWD(v));
    }
    auto from_error(nullopt_t) -> optional<T> {
        return {};
    }
};

```

```

template <class T, class E>
struct error_propagation_traits<expected<T, E>> {
    using value_type = T;
    using error_type = E;

    auto has_value(expected<T, E> const& e) -> bool {
        return e.has_value();
    }

    // extractors
    auto extract_value(auto&& e) -> auto&& {
        return *FWD(e);
    }
    auto extract_error(auto&& e) -> auto&& {
        return FWD(e).error();
    }

    // factories
    auto from_value(auto&& v) -> expected<T, E> {
        return expected<T, E>(in_place, FWD(v));
    }
    auto from_error(auto&& e) -> expected<T, E> {
        return expected<T, E>(unexpected, FWD(e));
    }
};

```

This also helps demonstrate the requirements for what `error_propagation_traits<O>` have to return:

- `has_value` is invoked on an lvalue of type `O` and returns `bool`
- `extract_value` takes some kind of `O` and returns a type that, after stripping qualifiers, is `value_type`
- `extract_error` takes some kind of `O` and returns a type that, after stripping qualifiers, is `error_type`
- `from_value` and `from_error` each returns an `O` (though their arguments need not be specifically a `value_type` or an `error_type`)

In the above case, `error_propagation_traits<expected<T, E>>::extract_error` will always give some kind of reference to `E` (either `E&`, `E const&`, `E&&`, or `E const&&`, depending on the value category of the argument), while `error_propagation_traits<optional<T>>::extract_error` will always be `std::nullopt_t`, by value. Both are fine, it simply depends on the type.

Since the extractors are only invoked on an `O` directly, you can safely assume that the object passed in is basically a forwarding reference to `O`, so `auto&&` is fine (at least pending something like [P2481R1](#)). The extractors have the implicit precondition that the object is in the state specified (e.g. `extract_value(o)` should only be called if `has_value(o)`, with the converse for `extract_error(o)`). The factories can accept anything though, and should probably be constrained.

The choice of desugaring based specifically on the return type (rather than relying on each object to produce some kind of construction disambiguator like `nullopt_t` or `unexpected<E>`) is not only that we can be more performant, but also we can allow conversions between different kinds of error types, which is useful when joining various libraries together:

```

auto foo(int i) -> tl::expected<int, E>;
auto bar(int i) -> std::expected<int, E>;

auto strcat(int i) -> Result<string, E>
{
    // this works
    return std::format("{}{}", foo(i)??, bar(i)??);
}

```

As long as each of these various error types opts into `error_propagation_traits` so that they can properly be constructed from an error, this will work just fine.

§ 4.2.1 Lifetime

Let's consider some function declarations, where `T`, `U`, `V`, and `E` are some well-behaved object types.

```

auto foo(T const&) -> V;
auto bar() -> std::expected<T, E>;
auto quux() -> std::expected<U, E>;

```

Now, consider the following fragment:

```
auto a = foo(bar()??);
```

The lifetime implications here should follow from the rest of the rules of the languages. Temporaries are destroyed at the end of the full-expression, temporaries bound to references do lifetime extension. In this case, `bar()` is a temporary of type `std::expected<T, E>`, which lasts until the end of the statement, `bar()??` gives you a `T&&` which refers into that temporary - which will be bound to the parameter of `foo()` - but that's safe because the `T` itself isn't going to be destroyed until the `std::expected<T, E>` is destroyed, which is after the call to `foo()` ends.

Note that this behavior is not really possible to express today using a statement rewrite. The inline macros for `bar()??` would do something like this:

```

auto a = foo(
    ({
        auto __tmp = bar();
        if (not __tmp) return std::move(__tmp).error();
        *__tmp;
        // __tmp destroyed here
    })
);

```

Using the statement-expression extension, the `std::expected<T, E>` will actually be destroyed *before* the call to `foo`, which means we have a dangling reference.

The coroutine rewrite wouldn't have this problem, for the same reason the suggested `bar()??` approach doesn't:

```
auto a = foo(co_await bar());
```

Now consider:

```
auto&& b = quux()??;
```

Here, extracting the value from `quux()` will give us a `U&&` that `b` binds to.

If this does not do lifetime extension, then the `std::expected<U, E>` is destroyed at the end of the statement. And we, once again, get a dangling reference. Note that this problem shows up either either of the macro propagation versions, all for the same reasons:

```

TRY(auto&& b, quux()); // dangles
auto&& b = TRY(quux()); // dangles

```

One way to avoid this issue is to have `extract_value`, when given an rvalue, return a temporary instead of an rvalue reference. This has performance implications though - you get an extra move that may not be necessary.

But a better way would be to recognize this pattern in the language itself, and allow lifetime extension for this case. Because we can recognize this situation (binding a reference to the result of `E??`), we probably should.

That is:

```

TRY(U&& a, quux()); // dangles
U&& b = TRY(quux()); // dangles
U&& c = quux()??;    // lifetime-extends the temporary quux()

```

Yet another advantage of the language feature.

§ 4.2.2 decltype

What does `decltype(E??)` evaluate to? Even though there's complex machinery going on here for actually propagating the error, the value type of `E??` itself isn't based on the return type of the function, it is based solely on `E`:

It is:

```
decltype(std::error_propagation_traits<std::remove_cvref_t<decltype(E)>>::extract_value(E))
```

As such, while `decltype(co_await E)` is ill-formed, `decltype(E??)` should be fine.

§ 4.2.3 requires

Consider this concept:

```
template <class T>
concept PropagatingError = requires (t t) { t??; }
```

With `decltype`, the type of `E??` is a function only of `E`. But in a broader context, the validity of the expression `E??` is based on both `E` and the return type of the function. For instance:

```
auto try_something() -> std::optional<int>;

auto f() -> std::optional<std::string> {
    return std::format("Got {}\n", try_something());
}

auto g() -> int {
    return try_something();
}

auto h() -> std::expected<int, std::string> {
    return try_something();
}
```

The usage in `f()` is fine, because both `optional<int>` and `optional<string>` opt in to `error_propagation_traits`, and `error_propagation_traits<optional<string>>::from_error(error_propagation_traits<optional<int>>::extract_error(try_something()))` is valid. Yes, that's a mouthful.

But `int` doesn't opt-in to `error_propagation_traits` at all, and while `std::expected<int, std::string>` does, its `from_error` would take a require something convertible to `std::string`, which `std::nullopt_t` is not. So both `g()` and `h()` must be ill-formed. Context is everything.

What does this say about what `PropagatingError<optional<int>>` should mean? I think it probably should be valid.

§ 4.2.4 Short-circuiting fold

One of the algorithms considered in the `ranges::fold` paper ([P2322R5]) was a short-circuiting fold. That paper ultimately didn't propose such an algorithm, since there isn't really a good way to generically write such a thing. Probably the best option in the paper was to have a mutating accumulation function that returns `bool` on failure?

But with this facility, there is a clear direction for how to write a generic, short-circuiting fold:

```
template <typename T>
concept PropagatingError = requires (T t) {
    typename error_propagation_traits<T>::value_type;
    typename error_propagation_traits<T>::error_type;

    { error_propagation_traits<T>::has_value(t) } -> boolean-testable;
    // etc. ...
};

template <input_iterator I,
        sentinel_for<I> S,
        class T,
        invocable<T, iter_reference_t<R>> F,
        PropagatingError Return = invoke_result_t<F&, T, iter_reference_t<R>>>
requires same_as<
    typename error_propagation_traits<Return>::value_type,
    T>
constexpr auto try_fold(I first, S last, T init, F accum) -> Ret
{
    for (; first != last; ++first) {
        init = std::invoke(accum,
```



```

        std::move(init),
        *first)??;
    }

    return error_propagation_traits<Ret>::from_value(std::move(init));
}

```

This `try_fold` can be used with an accumulation function that returns `optional<T>` or `expected<T, E>` or `boost::outcome::result<T>` or ... Any type that opts into being a `PropagatingError` will work.

Note that this may not be exactly the way we'd specify this algorithm, since we probably want to return something like a `pair<I, Ret>` instead, so the body wouldn't be able to use `??` and would have to go through `error_propagation_traits` manually for the error propagation. But that's still okay, since the important part was being able to have a generic algorithm to begin with.

§ 4.2.5 Range of expected to expected of Range

There's an algorithm in Haskell called `sequence` which takes a `t (m a)` and yields a `m (t a)`. In C++ terms, that might be an algorithm that takes a range of `expected<T, E>` and yields a `expected<vector<T>, E>` - which contains either all the results or the first error.

With the same `PropagatingError` concept from a above, this can be generalized to also work for `optional<T>` or any number of other `Result`-like types:

```

template <ranges::input_range R,
         PropagatingError T = remove_cvref_t<ranges::range_reference_t<R>>,
         typename Traits = error_propagation_traits<T>,
         typename Result = Traits::rebind<vector<typename Traits::value_type>>>
auto sequence(R&& r) -> Result
{
    vector<typename Traits::value_type> results;
    for (auto it = ranges::begin(r); it != ranges::end(r); ++it) {
        results.push_back((*it)??);
    }
    return Result::from_value(std::move(results));
}

```

§ 4.2.6 Naming

Because we don't have a proper language customization mechanism, we need to have two distinct things:

- a class template that contains all the functionality (which I'm naming here `std::error_propagation_traits`)
- a `concept` that checks if this class template is (a) specialized (b) correctly (which I'm naming here... `PropagatingError`, but I'm not sure that this name actually makes sense)

I think it's unfortunate that we need two different names for this, but that's the way of things at the moment. Also I have no idea what a good name for this `concept` is. Rust calls this `Try`, which [we wouldn't want](#). I'm open to suggestion.

§ 4.3 Potential directions to go from here

This paper is proposing just `??` and the machinery necessary to make that work (including a `concept`, opt-ins for `optional` and `expected`, but not the short-circuiting fold algorithm).

However, it's worth it for completeness to point out a few other directions that such an operator can take us.

§ 4.3.1 Error continuations

Several languages have a facility that allows for continuing to invoke member functions on optional values. This facility is called something different in every language (optional chaining in Swift, null-conditional operator in C#, safe call operator in Kotlin), but somehow it's all spelled the same and does the same thing anyway.

Given a `std::optional<std::string>` named `opt`, what that operator – spelled `?.` – means is approximately:

expression	C++ equivalent
<code>opt?.size()</code>	<code>opt.transform(&std::string::size) // technically UB</code>
<code>opt?.substr(from, to)</code>	<code>opt.transform([&](auto& s){ return s.substr(from, to); })</code>

Like the null coalescing meaning of `??` [described above](#), the semantics of `opt?.f()` can be achieved using library facilities today. The expression `E1?.E2`, if `E1` is an optional, basically means `E1.transform([&](auto&& e){ return FWD(e).E2; })`

Quite unlike `??`, there is a significant drop in readability and just the general nice-ness of the syntax.

The `error_propagation_traits` facility very nearly gives us the tools necessary to support such a continuation operator. Since what we need to do is:

- check if `E1` is truthy or falsey (`Traits::has_value(E1)`)
- extract the value of `E1` in order to perform the subsequent operation (`Traits::extract_value(E1).E2`)
- extract the error of `E1` in order to return early (`Traits::extract_error(E2)`)

We mostly need one more customization point: to put the types back together. What I mean is, consider:

```
auto f(int) -> std::expected<std::string, E>;

auto x = f(42)?.size();
```

The type of `x` needs to be `std::expected<size_t, E>`, since that's what the value case ends up being here. If we call that customization point `rebind`, as in:

```
template <typename T, typename E>
struct error_propagation_traits<expected<T, E>> {
    // ... rest as before ...

    template <class U>
    using rebind = expected<remove_cvref_t<U>, E>;
};
```

Then the above can be desugared into:

```
using _Traits = error_propagation_traits<remove_cvref_t<decltype(f(42))>>;
using _R = _Traits::rebind<decltype(_Traits::extract_value(f(42)).size())>;

auto&& e = f(42);
auto x = _Traits::has_value(e)
    ? error_propagation_traits<_R>::from_value(_Traits::extract_value(FWD(e)).size())
    : error_propagation_traits<_R>::from_error(_Traits::extract_error(FWD(e)));
```

That may seem like a mouthful. Because it is a mouthful. But it's a mouthful that the user doesn't have to write any part of, they just put `f(42)?.size()` and this does do the right thing.

At least, this mostly does the right thing. We still have to talk about copy elision. Consider this version:

```
struct X {
    auto f() -> std::mutex;
};

auto g() -> Result<X, E>;
auto n = g()?.f();
```

Presumably, `n` is a `Result<std::mutex, E>`, but in order for this to work, we can't just evaluate this as something like `Result<std::mutex, E>(g().value().f())`. `std::mutex` isn't movable.

The only way for this to work today is be able to pass a callable all the way through into this `Result`'s constructor. Which is to say, we desugar like so:

```
auto&& e = g();
auto n = _Traits::has_value(e)
    ? error_propagation_traits<_R>::from_value_func([&]() -> decltype(auto) {
        return _Traits::extract_value(FWD(e)).f();
    })
    : error_propagation_traits<_R>::from_error(_Traits::extract_error(FWD(e)));
```

By default, `error_propagation_traits<R>::from_value_func(f)` would just be `error_propagation_traits<R>::from_value(f())`.

This is weird, but it's something to think about.

Note also error continuation would only help in the member function case. If we want to continue into a non-member function, you'd need the sort of `.transform()` member function anyway.

§ 4.3.2 Not propagating errors

The `??` approach seems to work quite well at propagating errors: it's syntactically cheap, performant, and allows for integrating multiple libraries.

But what if we didn't want to propagate the error, but rather do something else with it? For `std::optional` and `std::expected`, we already have a UB-if-error accessor in the form of `*x` and a throw-if-error accessor in the form of `x.value()`. It seems like the corollary to an error-propagating `x??`

would be some sort of `x!!` that somehow forces the error differently.

While propagating the error only really has one way to go (you return it), there are quite a few different things you can do differently:

- `assert` that `has_value()`
- `abort()` if `not has_value()`
- `terminate()` if `not has_value()`
- `unreachable()` (or `[[assume]]`) if `not has_value()`
- `throw extract_error()` if `not has_value()`
- `throw f(extract_error())` if `not has_value()` for some `f`
- log an error and come up with some default value if `not has_value()`

That's a lot of different options, and the right one likely depends on context too.

An additional template parameter on the error type could drive what `x!!` does (as `Boost.Outcome` does, for instance), which would allow you to preserve the nice syntax if a particular error handling strategy is sufficiently common (maybe you always `throw`, so why would you want to write extra syntax for this case), but at a cost of suddenly having way more types. Although the `error_propagation_traits` approach does at least allow those “way more types” to interact well.

This behavior can be achieved by adding a new function to `error_propagation_traits` which desugars as follows:

<pre>auto val = expr!!;</pre>	<pre>auto&& __val = expr; using _Traits = std::error_propagation_traits< std::remove_cvref_t<decltype(__val)>>; if (not _Traits::has_value(__val)) { _Traits::fail(FWD(__val)); } auto val = _Traits::extract_value(FWD(__val));</pre>
-------------------------------	--

But this doesn't seem as valuable as `??` or even `?`, since this case is easy to add as a member function. Indeed, that's what `x.value()` and `*x` do for optional and expected (throw and undefined behavior, respectively).

Moreover, any of the kinds of behavior you want can be written as a free function:

```
template <class T, PropagatingError U = std::remove_cvref_t<T>>
auto narrow_value(T&& t) -> decltype(auto) {
    assert(std::error_propagation_traits<U>::has_value(t));
    return std::error_propagation_traits<U>::extract_value(FWD(t));
}

template <class T, PropagatingError U = std::remove_cvref_t<T>>
auto wide_value(T&& t) -> decltype(auto) {
    if (not std::error_propagation_traits<U>::has_value(t)) {
        [[unlikely]] throw std::error_propagation_traits<U>::extract_error(FWD(t));
    }
    return std::error_propagation_traits<U>::extract_value(FWD(t));
}

// etc.
```

Which further demonstrates the utility of the proposed facility.

§ 5 References

[P0323R12] Vicente Botet, JF Bastien, Jonathan Wakely. 2022-01-07. `std::expected`.

<https://wg21.link/p0323r12>

[P0709R4] Herb Sutter. 2019-08-04. Zero-overhead deterministic exceptions: Throwing values.

<https://wg21.link/p0709r4>

[P0779R0] Niall Douglas. 2017-10-15. Proposing operator `try()` (with added native C++ macro functions!).

<https://wg21.link/p0779r0>

[P2218R0] Marc Mutz. 2020-09-15. More flexible `optional::value_or()`.

<https://wg21.link/p2218r0>

[P2322R5] Barry Revzin. 2021-10-18. ranges::fold.

<https://wg21.link/p2322r5>

[P2481R1] Barry Revzin. 2022-07-15. Forwarding reference to specific type/template.

<https://wg21.link/p2481r1>

[P2505R4] Jeff Garland. 2022-06-17. Monadic Functions for std::expected.

<https://wg21.link/p2505r4>

[P2561R0] Barry Revzin. 2022-07-11. operator??

<https://wg21.link/p2561r0>

-
1. Outcome's TRY macro uses preprocessor overloading so void results don't get assigned e.g. TRY(auto x, expr) sets x to expr.value() while TRY(expr) ignores expr.value(). TRVA and TRYV *require* expr to have a value or for the value to be ignored respectively. One of them gets called by TRY() depending on argument count supplied.↵